ETW2001 Asg 1

AUTHOR
34477306 Ong Ying Tong

Assignment 1

Introduction

In this practice section, you will strengthen your understanding of R by applying basic data manipulation and logical operations to a real business dataset. You will practice extracting information from a data frame, performing calculations, filtering data using conditions, and summarising results to answer practical business questions. These activities are designed to build confidence in reading datasets, writing simple R expressions, and thinking logically about how data supports decision-making in real-world scenarios.

Load the Dataset

Task:

Load the CSV file and inspect its structure.

```
library(readxl)
```

```
Warning: package 'readxl' was built under R version 4.4.3
```

```
library(dplyr)
```

```
Warning: package 'dplyr' was built under R version 4.4.3
```

```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

    filter, lag

The following objects are masked from 'package:base':

    intersect, setdiff, setequal, union
```

```
library(tidyr)
```

```
Warning: package 'tidyr' was built under R version 4.4.3
```

```
orders_dt <- read_excel("superstore.xlsx")
head(orders_dt)
```

```
# A tibble: 6 × 27
  Category        City  Country Customer.ID Customer.Name Discount Market 记录数
  <chr>           <chr> <chr>   <chr>       <chr>            <dbl> <chr>   <dbl>
1 Office Supplies Los … United… LS-172304   Lycoris Saun…        0 US          1
```

```
2 Office Supplies Los … United… MV-174854   Mark Van Huff        0 US          1
3 Office Supplies Los … United… CS-121304   Chad Sievert         0 US          1
4 Office Supplies Los … United… CS-121304   Chad Sievert         0 US          1
5 Office Supplies Los … United… AP-109154   Arthur Prich…        0 US          1
6 Office Supplies Los … United… JF-154904   Jeremy Farry         0 US          1
# ℹ 19 more variables: Order.Date <dttm>, Order.ID <chr>, Order.Priority <chr>,
#   Product.ID <chr>, Product.Name <chr>, Profit <dbl>, Quantity <dbl>,
#   Region <chr>, Row.ID <dbl>, Sales <dbl>, Segment <chr>, Ship.Date <dttm>,
#   Ship.Mode <chr>, Shipping.Cost <dbl>, State <chr>, Sub.Category <chr>,
#   Year <dbl>, Market2 <chr>, weeknum <dbl>
```

```
#Interpretation:
#- Loads the dataset into a data frame.
#- Displays sample rows and data types.
```

QUESTION 1

```
#Count how many records each country has
top15_profit_per_shipping_unit <- orders_dt %>%
  group_by(Country) %>%
  mutate(
    count_country = n(),
    #Calculate the average shipping cost for each country
    avg_shipping_cost = mean(Shipping.Cost, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  #Keep only country with more than 200 records
  filter(
    count_country > 200,
    #Keep only transcations where shipping cost is above that country's average
    Shipping.Cost > avg_shipping_cost
  ) %>%
  mutate(
    #Create a new varaible porfit per shipping cost
    profit_per_shipping_unit = Profit / Shipping.Cost
  ) %>%
  #Rank from highest to lowest based on this new measure
  arrange(desc(profit_per_shipping_unit)) %>%
  select(
    #Only select the required columns
    Customer.Name,
    Product.Name,
    Country,
    Sales,
    Profit,
    Shipping.Cost,
    profit_per_shipping_unit
  ) %>%
  #Select top 15 records
  head(15)

top15_profit_per_shipping_unit
```

```
# A tibble: 15 × 7
   Customer.Name       Product.Name           Country Sales Profit Shipping.Cost
```

```
   <chr>              <chr>                  <chr>   <dbl>  <dbl>          <dbl>
 1 Tom Ashbrook       Canon imageCLASS 2200… United… 11200  3920.           46.0
 2 Pauline Johnson    Barricks Training Tab… Ukraine  4487  1436.           50.4
 3 Christopher Conant Canon PC1060 Personal… United…  2800   945.           35.0
 4 Tamara Chand       Canon imageCLASS 2200… United… 17500  8400.          349.
 5 Justin Hirsh       KitchenAid Refrigerat… Iraq     3172  1237.           53.6
 6 Maribeth Schnelling Samsung Smart Phone, … China    2545  1069.           52.3
 7 Michael Moore      Cisco Smart Phone, wi… Germany  2617  1151.           57.0
 8 Kelly Collister    Logitech P710e Mobile… United…  3347   636.           32.2
 9 Ben Peterman       Dania Classic Bookcas… Austra…  2596   923.           47.8
10 Brad Thomas        HP Fax Machine, Laser  Saudi …  1800   720.           38.5
11 Shirley Daniels    Fellowes PB500 Electr… United…  3813  1906.          103.
12 Katherine Nockton  HP Fax and Copier, Di… United…  1553   776.           42.6
13 Robert Marley      Office Star Executive… Colomb…   930   456.           26.3
14 Adam Bellavance    GBC DocuBind P400 Ele… United…  4355  1415.           82.7
15 Keith Herrera      Sharp AL-1530CS Digit… United…  1200   435.           26.0
# ℹ 1 more variable: profit_per_shipping_unit <dbl>
```

QUESTION 2

```r
#Keep only transactions with positive sales
#Then combin rows that belong to the same order
order_level_dt <- orders_dt %>%
  filter(Sales > 0) %>%
  group_by(Market, Year, Order.ID) %>%
  summarise(
    #Calculate total profit for each order
    order_profit = sum(Profit, na.rm = TRUE),
    #calculate the total sales for each order
    order_sales = sum(Sales, na.rm = TRUE),
    .groups = "drop"
  )
#Calculate the overall median profit using the order_level profit values
median_order_profit <- median(
  order_level_dt$order_profit,
  na.rm = TRUE
)

#Analyse each market-year combination using the order-level dataset
top10_consistent_markets <- order_level_dt %>%
  group_by(Market, Year) %>%
  mutate(
    number_of_orders = n(),
    #To count how many orders are in each market-year group
    average_profit_per_order = mean(order_profit, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  #Keep only market-year groups that have at least 50 orders
  filter(number_of_orders >= 50) %>%
  # Use groupby(market) to check consisitancy across all years
  group_by(Market) %>%
  #Filter markets whose yearly average profit per order
  # which never falls below the overall median order profit
```

```
  filter(min(average_profit_per_order, na.rm = TRUE) >= median_order_profit) %>%
  #For each remaning market, calculate averge sales across all remaining records
  summarise(
    average_sales = mean(order_sales, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  #Sort market frm highest to lowest average sales
  arrange(desc(average_sales)) %>%
  #Show the first 10 rows
  head(10)

top10_consistent_markets
```

```
# A tibble: 6 × 2
  Market average_sales
  <chr>          <dbl>
1 APAC            660.
2 EU              640.
3 US              459.
4 LATAM           421.
5 Canada          362.
6 Africa          351.
```

QUESTION 3

```
# For each sub-category, compute the typical discount level and keep only suspicious records
# Keep only record which the discount is higher than the median of the discount (typical discount)
suspicious_discount_dt <- orders_dt %>%
  group_by(Sub.Category) %>%
  mutate(
    typical_discount = median(Discount, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  filter(Discount > typical_discount) %>%
  mutate(
    #Return loss making if the profit is smaller than 0, else return non los making
    profit_label = ifelse(Profit < 0, "loss-making", "non-loss-making")
  )

# Only keep the market and segment which loss making share is gretaer than 40%
final_suspicious_discount_result <- suspicious_discount_dt %>%
  #Count how many suspicious record fall into each labled
  group_by(Market, Segment) %>%
  mutate(
    total_suspicious_records = n(),
    loss_making_records = sum(profit_label == "loss-making"),
    loss_making_share = loss_making_records / total_suspicious_records
  ) %>%
  filter(loss_making_share > 0.40) %>%
  group_by(Market, Segment, profit_label) %>%
  summarise(
    suspicious_record_count = n(),
    loss_making_share = max(loss_making_share),
```

```
    .groups = "drop"
  ) %>%
  #Sort the final result in desc order
  arrange(desc(loss_making_share), Market, Segment)

final_suspicious_discount_result
```

```
# A tibble: 30 × 5
   Market Segment     profit_label    suspicious_record_count loss_making_share
   <chr>  <chr>       <chr>                             <int>             <dbl>
 1 EMEA   Consumer    loss-making                         810             0.999
 2 EMEA   Consumer    non-loss-making                       1             0.999
 3 EMEA   Corporate   loss-making                         503             0.998
 4 EMEA   Corporate   non-loss-making                       1             0.998
 5 EMEA   Home Office loss-making                         293             0.997
 6 EMEA   Home Office non-loss-making                       1             0.997
 7 Africa Home Office loss-making                         191             0.979
 8 Africa Home Office non-loss-making                       4             0.979
 9 Africa Corporate   loss-making                         307             0.975
10 Africa Corporate   non-loss-making                       8             0.975
# ℹ 20 more rows
```

QUESTION 4

```
#LOOPING A (ORIGINAL DATASET)
# Initialized the cumulative profit to 0
cumulative_profit <- 0
#Loop through the dataset row by row in its original order
for (i in 1:nrow(orders_dt)) {
  #Skip the row which quantity is less than 2
  if (orders_dt$Quantity[i] < 2) next
  #Add the current row profit to the cummulative profit
  cumulative_profit <- cumulative_profit + orders_dt$Profit[i]
  #Stop the loop immediately once it exceed 25000
  if (cumulative_profit > 25000) {
    original_screening_result <- data.frame(
      screening_type = "Original dataset order",
      screening_position = i,
      order_id = orders_dt$Order.ID[i],
      customer_name = orders_dt$Customer.Name[i],
      cumulative_profit = cumulative_profit
    )
    break
  }
}

#LOOPING B (ARRANGE IN DESC ORDER)
# Reorder the dataset orders's sales from the highest to the lowest
orders_sales_sorted <- orders_dt[order(orders_dt$Sales, decreasing = TRUE), ]

#Reset the initialization for the second looping process
cumulative_profit <- 0
```

```r
#Skip the row if the quantity is less than 2
for (i in 1:nrow(orders_sales_sorted)) {
  if (orders_sales_sorted$Quantity[i] < 2) next

  cumulative_profit <- cumulative_profit + orders_sales_sorted$Profit[i]
  #Stop the loop once the running total first exceeds 25000
  if (cumulative_profit > 25000) {
    sales_sorted_screening_dt <- data.frame(
      screening_type = "Sales Sorted From Highest to Lowest",
      screening_position = i,
      order_id = orders_sales_sorted$Order.ID[i],
      customer_name = orders_sales_sorted$Customer.Name[i],
      cumulative_profit = cumulative_profit
    )
    break
  }
}

# Combine both stopping results to compare the two screening processes
comparison_of_stopping_points <- rbind(
  original_screening_result,
  sales_sorted_screening_dt
)

comparison_of_stopping_points
```

```
                     screening_type screening_position       order_id
1              Original dataset order                846 CA-2014-120936
2 Sales Sorted From Highest to Lowest                  6 CA-2013-117121
      customer_name cumulative_profit
1 Christine Abelman          25091.81
2     Adrian Barton          27215.22
```

QUESTION 5

```r
# Count how many countries each product is sold in
product_country_count_dt <- orders_dt %>%
  group_by(Product.Name, Country) %>%
  summarise(
    .groups = "drop"
  ) %>%
  group_by(Product.Name) %>%
  summarise(
    number_of_countries = n(),
    .groups = "drop"
  )

# Keep only products sold in at least 4 different countries
products_sold_in_four_countries_dt <- product_country_count_dt %>%
  filter(number_of_countries >= 4)

# For each product, calculate the median discount
# Afteep only products that satisfy the country requirement
```

```r
resilient_product_analysis_dt <- orders_dt %>%
  group_by(Product.Name) %>%
  mutate(
    median_discount = median(Discount, na.rm = TRUE)
  ) %>%
  ungroup() %>%
  filter(Product.Name %in% products_sold_in_four_countries_dt$Product.Name)

# Compare average profit under high-discount and low-or-equal-discount conditions
top5_resilient_products <- resilient_product_analysis_dt %>%
  group_by(Product.Name) %>%
  summarise(
    avg_profit_high_discount = mean(
      Profit[Discount > median_discount],
      na.rm = TRUE
    ),
    avg_profit_low_equal_discount = mean(
      Profit[Discount <= median_discount],
      na.rm = TRUE
    ),
    .groups = "drop"
  ) %>%

  # Create the profit gap between the two conditions
  mutate(
    profit_gap = avg_profit_high_discount - avg_profit_low_equal_discount
  ) %>%

  # Keep only products where the high-discount condition performs better
  filter(profit_gap > 0) %>%

  # Sort by the size of the gap and show the first 5 rows
  arrange(desc(profit_gap)) %>%
  head(5)


top5_resilient_products
```

```
# A tibble: 5 × 4
  Product.Name          avg_profit_high_disc…¹ avg_profit_low_equal…² profit_gap
  <chr>                                  <dbl>                  <dbl>      <dbl>
1 KitchenAid Microwave…                   484.                   79.5       405.
2 Cuisinart Refrigerat…                   351.                  148.        203.
3 Hoover Refrigerator,…                   331.                  142.        189.
4 Lesro Training Table…                  -428.                 -575.        148.
5 KitchenAid Stove, Si…                   541.                  413.        128.
# ℹ abbreviated names: ¹avg_profit_high_discount,
#   ²avg_profit_low_equal_discount
```

QUESTION 6

```r
# For each market, segment, and city combination,
# calculate the total sales contributed by that city
top10_market_segment_city_share <- orders_dt %>%
  group_by(Market, Segment, City) %>%
  summarise(
```

```
    city_sales_total = sum(Sales, na.rm = TRUE),
    .groups = "drop"
  ) %>%

  # Within each market-segment combination,
  # calculate the total sales and the city's share of those sales
  group_by(Market, Segment) %>%
  mutate(
    market_segment_sales_total = sum(city_sales_total, na.rm = TRUE),
    dominant_city_sales_share = city_sales_total / market_segment_sales_total
  ) %>%

  # Rank cities in desc order for the sales share within each market-segment combination
  arrange(desc(dominant_city_sales_share), .by_group = TRUE) %>%

  # Keep only the city with the greatest sales share in each market-segment combination
  filter(row_number() == 1) %>%
  ungroup() %>%

  # Keep only cases where the dominant city contributes more than one-third of the market-segment s
  filter(dominant_city_sales_share > 1/3) %>%

  # Show the selected columns only
  select(
    dominant_city = City,
    market = Market,
    segment = Segment,
    dominant_city_sales_share
  ) %>%

  # Sort the final result in desc order for the sales share
  arrange(desc(dominant_city_sales_share)) %>%

  # Show the first 10 records
  head(10)

top10_market_segment_city_share
```

```
# A tibble: 0 × 4
# ℹ 4 variables: dominant_city <chr>, market <chr>, segment <chr>,
#   dominant_city_sales_share <dbl>
```

QUESTION 7

```
# Higher Profit - increase the score
# Higher Shipping.Cost - reduce the score
# Higher Discount  - reduce the score
# Using Discount * Sales makes the discount penalty stronger when the sale amount is large

# Keep only orders placed in the final quarter of each year
orders_final_quarter <- orders_dt %>%
  mutate(
    order_month = as.numeric(format(Order.Date, "%m"))
  ) %>%
  filter(order_month %in% c(10, 11, 12))
```

```r
# Create a custom profitability score
orders_with_profitability_score <- orders_final_quarter %>%
  mutate(
    profitability_score = Profit - Shipping.Cost - (Discount * Sales)
  )

# Within each region and year, identify the top-scoring product
top_product_by_region_year <- orders_with_profitability_score %>%
  group_by(Region, Year, Product.Name) %>%
  summarise(
    total_profitability_score = sum(profitability_score, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  group_by(Region, Year) %>%
  arrange(desc(total_profitability_score), Product.Name, .by_group = TRUE) %>%
  filter(row_number() == 1) %>%
  ungroup()

# From those winners, determine which product appears most often across years within the same regic
region_product_frequency <- top_product_by_region_year %>%
  group_by(Region, Product.Name) %>%
  summarise(
    winning_count = n(),
    .groups = "drop"
  )

# Keep the most frequent winning product within each region
final_region_winner_report <- region_product_frequency %>%
  group_by(Region) %>%
  arrange(desc(winning_count), Product.Name, .by_group = TRUE) %>%
  filter(row_number() == 1) %>%
  ungroup()

# Save the report in a table
region_winner_report <- final_region_winner_report

View(region_winner_report)
```

QUESTION 8

```r
prio_order <- c("Critical", "High", "Medium", "Low")

# Compute the shipping gap and remove missing or impossible values
ship_gap <- orders_dt %>%
  mutate(
    days_taken = as.numeric(Ship.Date - Order.Date)
  ) %>%
  drop_na(Ship.Mode, Order.Priority, Order.Date, Ship.Date, days_taken) %>%
  filter(
    days_taken >= 0,
```

```r
    Order.Priority %in% prio_order
  )

# Compute average shipping days for each ship mode and priority
prio_avg <- ship_gap %>%
  mutate(
    Order.Priority = factor(Order.Priority, levels = prio_order, ordered = TRUE)
  ) %>%
  group_by(Ship.Mode, Order.Priority) %>%
  summarise(
    avg_days = mean(days_taken, na.rm = TRUE),
    .groups = "drop"
  ) %>%
  arrange(Ship.Mode, Order.Priority)

# Count how often a lower urgency level ships faster
# than the next higher urgency level within the same ship mode
ship_mode_summary <- prio_avg %>%
  group_by(Ship.Mode) %>%
  summarise(
    surprising_ship_mode_count = {
      surprising_count <- 0

      for (i in 1:(length(prio_order) - 1)) {
        higher_priority <- prio_order[i]
        lower_priority <- prio_order[i + 1]

        higher_priority_days <- avg_days[Order.Priority == higher_priority]
        lower_priority_days <- avg_days[Order.Priority == lower_priority]

        if (length(higher_priority_days) > 0 && length(lower_priority_days) > 0) {
          if (lower_priority_days < higher_priority_days) {
            surprising_count <- surprising_count + 1
          }
        }
      }

      surprising_count
    },
    .groups = "drop"
  ) %>%
  arrange(desc(surprising_ship_mode_count), Ship.Mode)

ship_mode_summary
```

```
# A tibble: 4 × 2
  Ship.Mode      surprising_ship_mode_count
  <chr>                               <dbl>
1 First Class                             1
2 Same Day                                1
3 Second Class                            0
4 Standard Class                          0
```

QUESTION 9

```r
# Summarise all orders into weekly totals within each year
weekly_sales_profit <- orders_dt %>%
  group_by(Year, weeknum) %>%
  summarise(
    total_weekly_sales = sum(Sales, na.rm = TRUE),
    total_weekly_profit = sum(Profit, na.rm = TRUE),
    .groups = "drop"
  )

# Ignore weeks where the total weekly profit is negative
non_negative_profit_weeks <- weekly_sales_profit %>%
  filter(total_weekly_profit >= 0)

# Get the list of years to process
year_list <- unique(non_negative_profit_weeks$Year)

# Create an empty table to store the final result
year_week_trigger_report <- data.frame(
  year = numeric(),
  triggering_week = numeric(),
  cumulative_sales = numeric(),
  weeks_counted_before_stopping = numeric()
)

# Process each year one by one
for (current_year in year_list) {

  # Keep only valid weeks for the current year and sort them by week number
  current_year_weeks <- non_negative_profit_weeks %>%
    filter(Year == current_year) %>%
    arrange(weeknum)

  # Calculate the sales threshold for the current year
  total_year_valid_sales <- sum(current_year_weeks$total_weekly_sales, na.rm = TRUE)
  trigger_sales_threshold <- 0.2 * total_year_valid_sales

  # Track cumulative sales and counted weeks
  cumulative_valid_sales <- 0
  valid_weeks_counted <- 0

  # Move through the weeks in order
  for (i in 1:nrow(current_year_weeks)) {
    cumulative_valid_sales <- cumulative_valid_sales + current_year_weeks$total_weekly_sales[i]
    valid_weeks_counted <- valid_weeks_counted + 1

    # Stop when cumulative sales first exceed the threshold
    if (cumulative_valid_sales > trigger_sales_threshold) {
      year_week_trigger_report <- rbind(
        year_week_trigger_report,
        data.frame(
          year = current_year,
          triggering_week = current_year_weeks$weeknum[i],
          cumulative_sales = cumulative_valid_sales,
          weeks_counted_before_stopping = valid_weeks_counted
        )
      )
      break
```

```
    }
  }
}

year_week_trigger_report
```

```
  year triggering_week cumulative_sales weeks_counted_before_stopping
1 2011              19           468656                            18
2 2012              18           550782                            17
3 2013              16           691520                            16
4 2014              17           903661                            17
```

QUESTION 10

```
state_subcategory_performance <- orders_dt %>%
  group_by(Region, State, Sub.Category) %>%
  summarise(
    number_of_records = n(),
    profitability_metric = ifelse(
      sum(Sales, na.rm = TRUE) == 0,
      NA,
      sum(Profit, na.rm = TRUE) / sum(Sales, na.rm = TRUE)
    ),
    shipping_cost_per_unit_sold = ifelse(
      sum(Quantity, na.rm = TRUE) == 0,
      NA,
      sum(Shipping.Cost, na.rm = TRUE) / sum(Quantity, na.rm = TRUE)
    ),
    .groups = "drop"
  ) %>%
  filter(number_of_records >= 30)

dt_avg_profitability <- mean(
  state_subcategory_performance$profitability_metric,
  na.rm = TRUE
)

dt_avg_shipping_per_unit <- mean(
  state_subcategory_performance$shipping_cost_per_unit_sold,
  na.rm = TRUE
)

top3_quiet_underperformers_by_region <- state_subcategory_performance %>%
  filter(
    profitability_metric < dt_avg_profitability,
    shipping_cost_per_unit_sold > dt_avg_shipping_per_unit
  ) %>%
  group_by(Region) %>%
  arrange(profitability_metric, .by_group = TRUE) %>%
  filter(row_number() <= 3) %>%
  ungroup() %>%
  select(
    Region,
    State,
    Sub.Category,
    number_of_records,
```

```
    profitability_metric,
    shipping_cost_per_unit_sold
  )

top3_quiet_underperformers_by_region
```

```
# A tibble: 15 × 6
   Region         State      Sub.Category number_of_records profitability_metric
   <chr>          <chr>      <chr>                     <int>                <dbl>
 1 Caribbean      Santo Dom… Chairs                       51               0.0242
 2 Central        Texas      Tables                       33              -0.141
 3 Central        Illinois   Chairs                       38              -0.108
 4 Central        Texas      Chairs                       61              -0.0947
 5 East           Ohio       Phones                       47              -0.190
 6 East           Pennsylva… Phones                       62              -0.183
 7 East           Pennsylva… Storage                      49              -0.122
 8 North          Distrito … Chairs                       40               0.0525
 9 Oceania        Queensland Bookcases                    47               0.0177
10 Southeast Asia National … Copiers                     31              -0.162
11 Southeast Asia National … Bookcases                   30              -0.138
12 Southeast Asia National … Phones                      35               0.0141
13 West           California Tables                      71              -0.00668
14 West           California Chairs                     130               0.0386
15 West           Washington Chairs                      34               0.0423
# ℹ 1 more variable: shipping_cost_per_unit_sold <dbl>
```